

DATA STRUCTURES – ASSIGNMENT REPORT (UNIT 4)

School of Engineering & Technology

Basics of Data Structures (ETCCBD201)

Lab Assignment – 4: Trees & Graphs

Student Details

- **Student Name:** Diya Kumal
 - **Roll Number:** 2501720001
 - **Section:** Bsc Computer Science
-

INDEX

1. Introduction
 2. Objectives
 3. Task 1 – Binary Search Tree (BST)
 - Introduction to BST
 - BST Operations
 - Time Complexity
 - Inorder Traversal Explanation
 - Observations
 4. Task 2 – Graph Representation using Adjacency List
 - Introduction to Graphs
 - Adjacency List Representation
 - BFS Traversal
 - DFS Traversal
 - BFS vs DFS
 - Real World Applications
 - Observations
 5. Output Screenshots / Console Output
 6. Conclusion
 7. References
-

1. INTRODUCTION

Data structures are used to organize and store data efficiently so that operations such as insertion, deletion, searching, and traversal can be performed quickly. Different data structures are used for different purposes depending on the requirement of the application.

In this assignment, Binary Search Tree (BST) and Graph data structures are implemented using Python. The assignment demonstrates how BST operations work and how graph traversal algorithms such as Breadth First Search (BFS) and Depth First Search (DFS) are performed using adjacency list representation.

2. OBJECTIVES

The objectives of this assignment are:

- To understand the working of Binary Search Trees.
 - To perform insertion, deletion, searching, and traversal operations in BST.
 - To understand graph representation using adjacency lists.
 - To implement BFS and DFS traversal algorithms.
 - To analyze the time complexity of different operations.
 - To understand real-world applications of trees and graphs.
-

3. TASK 1 – BINARY SEARCH TREE (BST)

3.1 Introduction to BST

A Binary Search Tree (BST) is a hierarchical data structure where each node contains a key and satisfies the following properties:

- All values in the left subtree are smaller than the root node.
- All values in the right subtree are greater than the root node.
- Both left and right subtrees are also Binary Search Trees.

BST allows efficient searching, insertion, and deletion operations.

3.2 BST Operations Implemented

The following operations were implemented in Python:

1. Insert Operation

The insert operation places a new node in the correct position according to BST rules.

2. Search Operation

The search operation checks whether a particular key exists in the BST.

3. Delete Operation

Deletion in BST handles three cases:

Case 1: Node with No Child (Leaf Node)

The node is directly removed.

Example:

- Delete node 20

Case 2: Node with One Child

The child node replaces the deleted node.

Example:

- Insert 65
- Delete 60

Case 3: Node with Two Children

The inorder successor replaces the deleted node.

Example:

- Delete node 30 or 50

4. Inorder Traversal

In inorder traversal:

- Left subtree is visited first
- Root node is visited second
- Right subtree is visited last

This traversal prints BST elements in sorted order.

3.3 BST Test Data Used

Inserted Elements

50, 30, 70, 20, 40, 60, 80

Search Operations

- Search 20 → Found
- Search 90 → Not Found

Delete Operations

- Delete 20 (Leaf Node)
- Insert 65 and Delete 60 (One Child)

- Delete 30 (Two Children)
-

3.4 Why Inorder Traversal Prints Sorted Order

In a BST, all values in the left subtree are smaller than the root, and all values in the right subtree are greater than the root.

Inorder traversal follows the sequence:

Left → Root → Right

Because smaller values are visited before the root and larger values are visited afterward, the output becomes sorted automatically.

Example:

20 30 40 50 60 70 80

3.5 Time Complexity of BST Operations

Operation	Average Case	Worst Case
Insert	$O(\log n)$	$O(n)$
Search	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$
Traversal	$O(n)$	$O(n)$

Explanation

- In balanced BSTs, operations take $O(\log n)$ time because the tree height remains small.
 - In skewed trees, operations may take $O(n)$ because the tree behaves like a linked list.
-

3.6 Observations

- BST allows efficient searching and insertion.
 - Inorder traversal always prints data in sorted order.
 - Deletion is the most complex operation because it requires handling multiple cases.
 - Performance depends on the balance of the tree.
-

4. TASK 2 – GRAPH REPRESENTATION USING ADJACENCY LIST

4.1 Introduction to Graphs

A graph is a non-linear data structure consisting of:

- Vertices (Nodes)
- Edges (Connections)

Graphs are widely used in computer networks, maps, social media, routing systems, and search engines.

In this assignment, a directed weighted graph was implemented using adjacency lists.

4.2 Adjacency List Representation

An adjacency list stores graph data as a list where each node contains its connected neighbors and edge weights.

Graph Used

Nodes

A, B, C, D, E, F

Directed Weighted Edges

- $A \rightarrow B$ (2)
 - $A \rightarrow C$ (4)
 - $B \rightarrow D$ (7)
 - $B \rightarrow E$ (3)
 - $C \rightarrow E$ (1)
 - $D \rightarrow F$ (5)
 - $E \rightarrow D$ (2)
 - $E \rightarrow F$ (6)
 - $C \rightarrow F$ (8)
-

4.3 Adjacency List Output

A : [(B,2), (C,4)]

B : [(D,7), (E,3)]

C : [(E,1), (F,8)]

D : [(F,5)]

E : [(D,2), (F,6)]

F : []

4.4 Breadth First Search (BFS)

BFS visits nodes level by level.

It uses a queue data structure.

BFS Traversal from A

A → B → C → D → E → F

Working of BFS

1. Start from the source node.
 2. Visit all neighboring nodes.
 3. Continue visiting nodes level by level.
 4. Use a queue to manage traversal order.
-

4.5 Depth First Search (DFS)

DFS visits nodes deeply before backtracking.

It uses recursion or stack.

DFS Traversal from A

A → B → D → F → E → C

Working of DFS

1. Start from the source node.
 2. Visit one path completely.
 3. Backtrack when no unvisited nodes remain.
 4. Continue until all nodes are visited.
-

4.6 Difference Between BFS and DFS

BFS	DFS
Uses Queue	Uses Stack/Recursion
Visits level by level	Visits depth wise
Finds shortest path in unweighted graph	Useful for cycle detection
Requires more memory	Requires less memory
Easier for shortest path problems	Better for deep traversal

4.7 Real World Applications

BFS Applications

- Shortest path finding
- GPS navigation systems
- Network broadcasting
- Social networking applications

DFS Applications

- Cycle detection
 - Topological sorting
 - Maze solving
 - File system traversal
-

4.8 Observations

- Adjacency list efficiently stores sparse graphs.
 - BFS is useful when nearest nodes must be explored first.
 - DFS is useful when complete path exploration is required.
 - Graph traversal algorithms are important in networking and artificial intelligence.
-

5. OUTPUT SCREENSHOTS / CONSOLE OUTPUT

BST Output

Inserted Elements: 50 30 70 20 40 60 80

Search Results:

- 20 Found
- 90 Not Found

Inorder Traversal: 20 30 40 50 60 70 80

After Deleting 20: 30 40 50 60 70 80

After Inserting 65 and Deleting 60: 30 40 50 65 70 80

After Deleting 30: 40 50 65 70 80

Graph Output

Adjacency List:

A : [(B,2), (C,4)] B : [(D,7), (E,3)] C : [(E,1), (F,8)] D : [(F,5)] E : [(D,2), (F,6)] F : []

BFS Traversal from A: A B C D E F

DFS Traversal from A: A B D F E C

6. CONCLUSION

In this assignment, Binary Search Tree and Graph data structures were successfully implemented using Python.

BST operations such as insertion, searching, deletion, and inorder traversal were performed successfully. Different deletion cases were tested and verified.

Graph representation using adjacency lists was also implemented successfully. BFS and DFS traversal algorithms were executed correctly from the starting node.

This assignment improved understanding of tree structures, graph traversal techniques, and their practical applications in computer science.

7. REFERENCES

1. Python Official Documentation – <https://docs.python.org/>
 2. GeeksforGeeks – <https://www.geeksforgeeks.org/>
 3. Introduction to Algorithms by Cormen
 4. Data Structures using Python Notes
-